

DINDIN

Documentacao tecnica para manutencao

Este documento descreve a arquitetura atual do sistema, modelo de dados, regras de negocio, fluxos principais da API e orientacoes praticas para manutencao evolutiva e corretiva.

Versao do documento: 2026-03-21

RESUMO

Stack: Vue 3 via CDN, Node.js sem framework, SQLite local.

Entrada HTTP: server.js -> src/server.js

Persistencia: data/gastos.sqlite

Modelo: multiusuario local, sessao em memoria, API REST simples.

1. Objetivo do sistema

O Dindin e uma aplicacao local de controle financeiro mensal focada em contas recorrentes. O sistema permite manter um cadastro de gastos, materializar esses gastos em cada mes salvo, editar valores e status por mes e acompanhar o historico de cada usuario.

2. Arquitetura geral

Camadas do backend

- src/server.js : inicializa schema, sobe o servidor HTTP e distribui entre API e arquivos estaticos.
- src/api/handleApi.js : roteamento manual das rotas REST e orquestracao dos casos de uso.
- src/services/* : regras de dominio e fluxos compostos, como bootstrap, meses, e-mail e sessao.
- src/repositories/* : acesso SQL organizado por agregado.
- src/db/* : conexao SQLite e inicializacao/migracao de schema.
- src/lib/* : validacoes, serializacao, utilitarios HTTP, conversoes e seguranca.

Camadas do frontend

- public/index.html : shell da pagina e template do app Vue.
- public/app.js : estado, metodos e fluxo de navegacao da SPA.
- public/styles.css : estilo global, responsividade e temas claro/escuro.
- Sem build step. O Vue 3 e carregado via CDN.

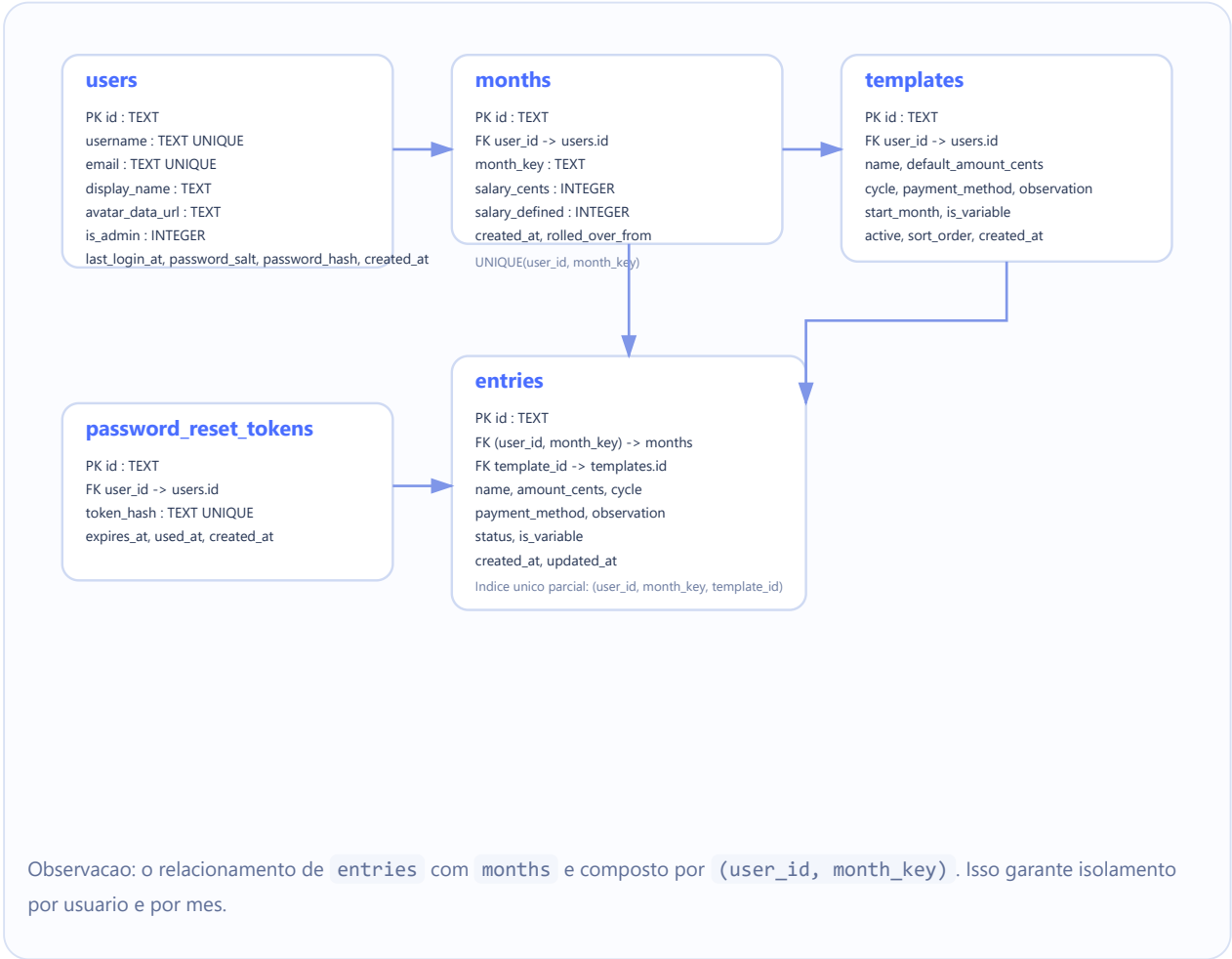
3. Estrutura de diretorios

Caminho	Responsabilidade
server.js	Ponto de entrada fino que delega para src/server.js .
src/api	Roteamento HTTP da API.

Caminho	Responsabilidade
src/db	Conexao SQLite, schema e transacoes.
src/repositories	Consultas e comandos SQL por entidade.
src/services	Casos de uso compostos e regras de negocio.
src/lib	Funcoes utilitarias compartilhadas.
public	Frontend estatico servido pelo backend.
data/gastos.sqlite	Banco local persistido em arquivo.

4. Modelo de dados (UML / ER)

O banco trabalha com cinco entidades principais. A relacao central e: um `user` possui muitos `months`, `templates` e `entries`. Os `templates` servem de base para gerar `entries` por mes.



5. Regras de negocio principais

Usuarios e autenticao

- Existe um usuario admin padrao local: `admin / admin123` se o banco nascer vazio.
- As sessoes sao mantidas em memoria no processo Node. Reiniciar o servidor invalida todas as sessoes.
- Senhas sao hashadas com `bcrypt`.
- Fluxo de redefinicao de senha usa token hash em SQLite, com expiracao de 60 minutos.
- Deve sempre existir ao menos um administrador.

Meses

- O mes e identificado por `YYYY-MM`.
- Um mes e persistido quando necessario, principalmente ao salvar salario ou criar cadastro no mes.
- Ao criar um novo mes salvo, o sistema materializa nele os templates fixos ja validos para aquele periodo.
- Ao excluir um mes, seus lancamentos tambem sao excluidos.
- Se um template comecava no mes excluido, o `start_month` e empurrado para o proximo mes para evitar recriacao indevida.

Templates (cadastros)

- **Template fixo:** pode ser copiado para meses futuros salvos.
- **Template variavel:** nao e autopropagado para meses futuros se o lancamento ainda nao existir.
- `cycle` aceita apenas `Inicio Do Mes` ou `Quinzena`.
- Um template pode ser desativado; isso nao remove historico passado fora do mes atual deletado no fluxo da API.

Entries (lancamentos)

- Todo entry pertence a um usuario e um mes.
- Status permitido: `pending`, `paid`, `saved`.
- Existe deduplicacao defensiva de lancamentos derivados de template.
- Os valores sao armazenados em centavos para evitar problemas de precisao.

6. Fluxos tecnicos relevantes

6.1 Bootstrap da aplicacao

- O frontend chama `GET /api/session` para verificar autenticao.
- Quando autenticado, chama `GET /api/bootstrap` com ou sem `?month=YYYY-MM`.
- O backend monta um payload consolidado com usuario, mes ativo, resumo, lista de meses, templates e entries.
- Se o mes solicitado ja existir, o backend ainda garante que os templates fixos validos estejam materializados naquele mes.

6.2 Criacao de cadastro

- Rota: `POST /api/templates`.
- Valida campos, garante a existencia do mes e grava o template.
- Cria o entry correspondente no mes atual.
- Se o template for fixo, tambem pode gerar entries em meses futuros ja persistidos.

6.3 Edicao de cadastro

- Rota: `PATCH /api/templates/:id`.
- Atualiza atributos do template e sincroniza o entry correspondente no mes selecionado.
- Se o template ficar invalido para aquele mes por causa do `start_month`, o entry pode ser removido.

6.4 Exclusao de mes

- Rota: `DELETE /api/months/:monthKey` .
- Executa transacao SQLite.
- Move o `start_month` dos templates que nasciam naquele mes para o mes seguinte.
- Remove entries e o proprio mes.

7. Mapa resumido da API

Rota	Metodo	Uso
<code>/api/session</code>	GET	Verifica sessao atual.
<code>/api/login</code>	POST	Autenticacao.
<code>/api/register</code>	POST	Cadastro de novo usuario.
<code>/api/logout</code>	POST	Encerra sessao.
<code>/api/bootstrap</code>	GET	Retorna estado consolidado do app.
<code>/api/salary</code>	POST	Salva salario do mes.
<code>/api/templates</code>	POST	Cria cadastro.
<code>/api/templates/:id</code>	PATCH / DELETE	Edita ou desativa cadastro.
<code>/api/entries/:id</code>	PATCH / DELETE	Edita ou exclui lancamento.
<code>/api/profile</code>	PATCH / POST	Edita perfil.
<code>/api/change-password</code>	POST	Troca senha autenticada.
<code>/api/password-reset/*</code>	GET / POST	Fluxo de redefinicao de senha.
<code>/api/admin/users</code>	GET / POST / PATCH	Administracao de contas locais.

8. Convencoes e pontos de manutencao

Padrao atual: manter regras de negocio em `services` , SQL em `repositories` e validacoes/com conversoes em `lib` . Evitar reintroduzir regras diretamente em `handleApi.js` alem da orquestracao da rota.

- Ao adicionar coluna nova, preferir migracao incremental em `src/db/schema.js` antes de qualquer rebuild destrutivo.
- Ao criar regra de negocio nova para mes/template/entry, avaliar impacto em `buildBootstrapPayload` porque o frontend depende fortemente desse payload unico.
- Ao alterar contratos da API, revisar tambem `public/app.js` .
- Como a sessao e em memoria, qualquer estrategia de multi-instancia exigira armazenamento persistente de sessao.
- Como o frontend usa Vue via CDN e template em HTML, nao ha pipeline de build. Isso simplifica deploy local, mas aumenta a necessidade de disciplina manual em organizacao do codigo.

9. Checklist de manutencao segura

- Subir a aplicacao e testar login/logout.
- Validar criacao e edicao de cadastro fixo e variavel.

- Validar edicao de status e valor de lancamento.
- Testar troca de mes e exclusao de mes.
- Testar modo admin, alteracao de perfil e troca de senha.
- Se houver SMTP configurado, testar tambem o reset de senha.

Limitacoes atuais: a documentacao reflete o estado do codigo em 2026-03-21. Sessoes em memoria, Vue via CDN e uso de `node:sqlite` experimental no Node seguem sendo decisoes conscientes de simplicidade para um sistema local.

Documento gerado automaticamente para o projeto Dindin. Finalidade: manutencao tecnica, onboarding e apoio a evolucoes futuras.